



CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL - SNIES 2828

Guía paso a paso de pruebas unitarias

Programa: Ingeniería De Sistemas

Asignatura: Calidad De Software

Profesor: Jose Miguel llanos Mosquera

Integrantes

María Sofia Aljure Herrera

María Juliana Ferro Bonilla

Jose Miguel Vera Garzón

Corporación Universitaria Del Huila Corhuila

Neiva – Huila

Abril 2026



CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL - SNIES 2828

Contenido

1. Introducción.....	3
2. Objetivo general	3
3. Objetivos específicos.....	3
4. Marco teórico.....	4
5. Desarrollo	5
6. Herramientas y Tecnologías:.....	5
7. Configuración del Proyecto:	6
8. Tablas de requisitos:.....	7
9. Tablas de casos de prueba:	8
10. Implementación de la Clase Principal:.....	9
11. Pruebas Unitarias:	11
12. Ejecución de Pruebas y Cobertura:	12
13. Evaluación de Calidad:.....	13
14. Documentación JavaDoc:.....	14
15. Conclusiones:	15
16. Referencias:	16
Tabla 1 Tabla de requisitos.....	8
Tabla 2 Tabla de casos de prueba.....	9
Tabla 3 Resumen de Cobertura	13
Ilustración 1 Resultados Obtenidos	13
Ilustración 2 Resultados Obtenidos	13
Ilustración 3 Documentación JavaDoc	14



1. Introducción

En el desarrollo de software, garantizar la calidad del producto es un aspecto fundamental para asegurar su correcto funcionamiento y confiabilidad. Las pruebas unitarias permiten verificar que cada componente del sistema opere de manera adecuada, detectando errores en etapas tempranas del desarrollo.

En este proyecto se implementa un sistema de conversión de temperaturas entre grados Celsius y Fahrenheit, el cual sirve como caso práctico para aplicar conceptos de calidad de software. A través del uso de herramientas como JUnit y JaCoCo, se busca validar el correcto funcionamiento del sistema, asegurando precisión en los resultados, manejo de errores y cumplimiento de los requisitos establecidos.

2. Objetivo general

Desarrollar y validar un sistema de conversión de temperaturas mediante la aplicación de pruebas unitarias, garantizando la calidad del software a través del cumplimiento de requisitos funcionales y métricas de cobertura.

3. Objetivos específicos

- Implementar un sistema que permita la conversión de temperaturas entre Celsius y Fahrenheit.
- Diseñar y ejecutar casos de prueba utilizando JUnit para validar el correcto funcionamiento del sistema.
- Evaluar la calidad del software mediante métricas de cobertura utilizando JaCoCo.

4. Marco teórico

- **Calidad de Software**

La calidad de software se define como el conjunto de características que permiten que un sistema cumpla con los requisitos establecidos y satisfaga las necesidades del usuario. Según la International Organization for Standardization, la calidad incluye atributos como funcionalidad, confiabilidad, usabilidad, eficiencia, mantenibilidad y portabilidad.

Un software de calidad no solo debe funcionar correctamente, sino también ser robusto ante errores, fácil de mantener y eficiente en su ejecución.

- **Pruebas Unitarias**

Las pruebas unitarias son un tipo de prueba de software que se enfoca en validar las unidades más pequeñas del sistema, como funciones o métodos individuales. De acuerdo con Roger S. Pressman, estas pruebas permiten detectar errores en etapas tempranas del desarrollo, reduciendo costos y mejorando la calidad del producto final.

Las pruebas unitarias se caracterizan por ser automatizadas, rápidas y repetibles, lo que facilita su ejecución continua durante el desarrollo.

- **Framework JUnit**

JUnit es un framework ampliamente utilizado en el desarrollo de aplicaciones Java para la creación de pruebas unitarias. Permite verificar el comportamiento esperado del código mediante aserciones, como `assertEquals` y `assertThrows`.

Según la JUnit Team, JUnit facilita la automatización de pruebas y la integración con herramientas de construcción como Maven, contribuyendo a mejorar la calidad del software.



- **Cobertura de Código con JaCoCo**

JaCoCo es una herramienta que permite medir la cobertura de código en proyectos Java, indicando qué partes del programa han sido ejecutadas durante las pruebas. Esta métrica es fundamental para evaluar la efectividad de las pruebas unitarias.

De acuerdo con la Eclipse Foundation, JaCoCo proporciona información detallada sobre cobertura de líneas, ramas y métodos, ayudando a identificar áreas del código que requieren más pruebas.

- **JavaDoc**

JavaDoc es una herramienta estándar de Java que permite generar documentación automática a partir de comentarios estructurados en el código fuente. Facilita la comprensión del funcionamiento de clases y métodos, mejorando la mantenibilidad del software.

Según Oracle Corporation, JavaDoc permite documentar parámetros, valores de retorno y excepciones, generando documentación en formato HTML que puede ser utilizada por otros desarrolladores.

5. Desarrollo

6. Herramientas y Tecnologías:

Para la implementación y pruebas del sistema de conversión de temperaturas se utilizaron las siguientes herramientas:

IntelliJ IDEA: IDE para desarrollo en Java.

Java 11: Lenguaje de programación principal.



Maven: Gestión de dependencias y compilación.

JUnit 5 (Jupiter): Framework para pruebas unitarias.

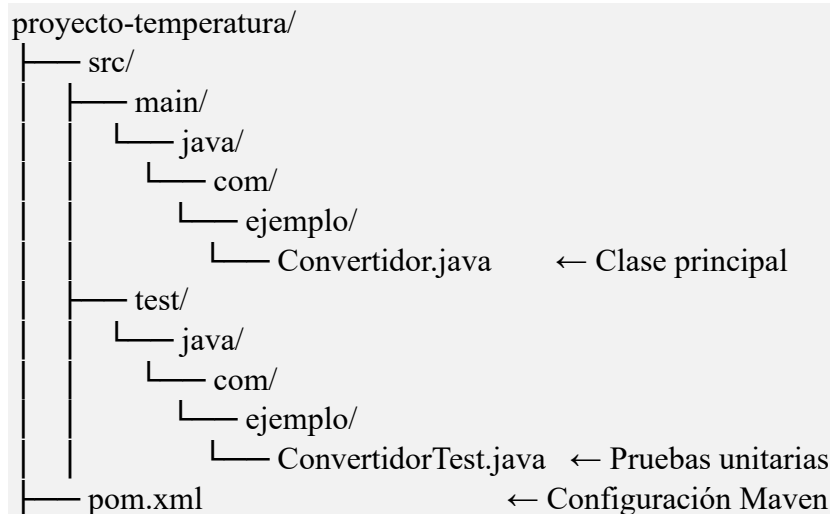
JaCoCo: Métrica de cobertura de código.

Git: Control de versiones.

JavaDoc: Generación de documentación automática.

7. Configuración del Proyecto:

Estructura de Carpetas



Archivo pom.xml (Dependencias)

```
<project>
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.ejemplo</groupId>
  <artifactId>convertidor-temperatura</artifactId>
  <version>1.0.0</version>

  <properties>
    <maven.compiler.source>11</maven.compiler.source>
    <maven.compiler.target>11</maven.compiler.target>
  </properties>
```



```
<dependencies>
  <!-- JUnit 5 (Jupiter) -->
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.9.3</version>
    <scope>test</scope>
  </dependency>
</dependencies>

<build>
  <plugins>
    <!-- Maven Surefire (Ejecutar pruebas) -->
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>2.22.2</version>
    </plugin>

    <!-- JaCoCo (Cobertura de Código) -->
    <plugin>
      <groupId>org.jacoco</groupId>
      <artifactId>jacoco-maven-plugin</artifactId>
      <version>0.8.10</version>
      <executions>
        <execution>
          <goals><goal>prepare-agent</goal></goals>
        </execution>
        <execution>
          <id>report</id>
          <phase>test</phase>
          <goals><goal>report</goal></goals>
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>
</project>
```

8. Tablas de requisitos:



ID	Requisito Funcional
RF1	El sistema debe convertir temperaturas de Celsius a Fahrenheit
RF2	El sistema debe convertir temperaturas de Fahrenheit a Celsius
RF3	El sistema debe validar que los valores ingresados sean numéricos
RF4	El sistema debe permitir el manejo de valores negativos
RF5	El sistema debe garantizar precisión en los resultados (decimales correctos)

Tabla 1 Tabla de requisitos

9. Tablas de casos de prueba:

ID Caso	Requisito	Descripción del Caso de Prueba	Entrada	Resultado Esperado
CP-001	RF-001	Conversión básica de Celsius a Fahrenheit	0°C	32°F
CP-002	RF-001	Conversión con valor positivo	25°C	77°F
CP-003	RF-002	Conversión básica de Fahrenheit a Celsius	32°F	0°C
CP-004	RF-002	Conversión con valor positivo	68°F	20°C
CP-005	RF-003	Validar entrada no numérica	"abc"	Error / rechazo de entrada
CP-006	RF-003	Validar entrada vacía	" "	Error / mensaje de campo obligatorio



CP-007	RF-004	Conversión de valor negativo en Celsius	-10°C	14°F
CP-008	RF-004	Conversión de valor negativo en Fahrenheit	-4°F	-20°C
CP-009	RF-005	Conversión con precisión decimal	37°C	98.6°F
CP-010	RF-005	Conversión con decimales largos	36.5°C	97.7°F

Tabla 2 Tabla de casos de prueba

10. Implementación de la Clase Principal:

```
package com.corhuila.calidad;
```

```
/**
```

```
 * Clase encargada de realizar conversiones de temperatura entre
```

```
 * grados Celsius y Fahrenheit.
```

```
 * <p>
```

```
 * Soporta valores negativos y precisión decimal.
```

```
 * </p>
```

```
 *
```

```
 * @version 1.0
```

```
 */
```

```
public class Convertidor {
```

```
 /**
```

```
 * Convierte una temperatura de grados Celsius a Fahrenheit.
```

```
 * <p><b>Requerimiento:</b> RF-001</p>
```

```
 *
```

```
 * @param c temperatura en grados Celsius
```



CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL - SNIES 2828

* @return temperatura convertida a grados Fahrenheit

*/

```
public double celsiusAFahrenheit(double c) {
```

```
    return (c * 9/5) + 32;
```

```
}
```

```
/**
```

* Convierte una temperatura de grados Fahrenheit a Celsius.

* <p>Requerimiento: RF-002</p>

* @param f temperatura en grados Fahrenheit

* @return temperatura convertida a grados Celsius

*/

```
public double fahrenheitACelsius(double f) {
```

```
    return (f - 32) * 5/9;
```

```
}
```

```
/**
```

* Verifica si una cadena de texto representa un valor numérico válido.

* <p>Requerimiento: RF-003</p>

* @param input cadena de texto a validar

* @return {@code true} si el valor es numérico, {@code false} en caso contrario

*/

```
public boolean esNumerico(String input) {
```

```
    if (input == null || input.trim().isEmpty()) return false;
```

```
    try {
```

```
        Double.parseDouble(input);
```

```
        return true;
```

```
    } catch (NumberFormatException e) {
```

```
        return false;
```

```
}
```





CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL - SNIES 2828

11. Pruebas Unitarias:

```
import com.corhuila.calidad.Convertidor;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
/**
 * Clase de pruebas unitarias para la clase {@link Convertidor}.
 * <p>
 * Verifica el correcto funcionamiento de las conversiones de temperatura
 * y la validación de entradas numéricas.
 * </p>
 * @version 1.0
 */
public class ConvertidorTest {
    /**
     * Instancia del convertidor utilizada en las pruebas.
     */
    private Convertidor conv = new Convertidor();
    /**
     * Prueba la conversión de Celsius a Fahrenheit.
     * <p><b>Requerimiento:</b> RF-001</p>
     */
    @Test
    public void testCelsiusAFahrenheit() {
        assertEquals(32.0, conv.celsiusAFahrenheit(0), 0.1);
        assertEquals(77.0, conv.celsiusAFahrenheit(25), 0.1);
        assertEquals(14.0, conv.celsiusAFahrenheit(-10), 0.1);
        assertEquals(98.6, conv.celsiusAFahrenheit(37), 0.1);
        assertEquals(97.7, conv.celsiusAFahrenheit(36.5), 0.1);
    }
    /**
     * Prueba la conversión de Fahrenheit a Celsius.
     * <p><b>Requerimiento:</b> RF-002</p>
     */
    @Test
    public void testFahrenheitACelsius() {
        assertEquals(0.0, conv.fahrenheitACelsius(32), 0.1);
        assertEquals(20.0, conv.fahrenheitACelsius(68), 0.1);
        assertEquals(-20.0, conv.fahrenheitACelsius(-4), 0.1);
    }
}
```





/**

* Prueba la validación de entradas numéricas.

* <p>Requerimiento: RF-003</p>

*/

@Test

```
public void testEsNumerico() {  
    assertTrue(conv.esNumerico("25"));  
    assertTrue(conv.esNumerico("-10"));  
    assertFalse(conv.esNumerico("abc"));  
    assertFalse(conv.esNumerico("  "));  
}  
}
```

12. Ejecución de Pruebas y Cobertura:

- Ejecutar todas las pruebas:

```
mvn test
```

- Resultado obtenido:

T E S T S

Running ConvertidorTest

Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.062 s - in ConvertidorTest

Results:

Tests run: 3, Failures: 0, Errors: 0, Skipped: 0

--- jacoco:0.8.10:report (report) @ convertidor-temperatura ---

Loading execution data file C:\Users\veraj\OneDrive\Escritorio\UNIVERSIDAD\1-CALIDAD_SOFTWARE\Conversion\Conversortemp\target\jacoco.exec

Analyzed bundle 'convertidor-temperatura' with 1 classes

BUILD SUCCESS



Total, time: 4.612 s

- Ejecutar pruebas + generar reporte JaCoCo:

mvn clean test jacoco:report

- Resultado obtenido

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
Convertidor		100 %		75 %	1	6	0	8	0	4	0	1
Total	0 of 35	100 %	1 of 4	75 %	1	6	0	8	0	4	0	1

Ilustración 1 Resultados Obtenidos

Convertidor

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
esNumerico(String)		100 %		75 %	1	3	0	5	0	1
celsiusAFahrenheit(double)		100 %		n/a	0	1	0	1	0	1
fahrenheitACelsius(double)		100 %		n/a	0	1	0	1	0	1
Convertidor()		100 %		n/a	0	1	0	1	0	1
Total	0 of 35	100 %	1 of 4	75 %	1	6	0	8	0	4

Ilustración 2 Resultados Obtenidos

- Resumen de Cobertura (Esperado)

Métrica	Resultado	Evaluación
Line Coverage	100%	Excelente
Branch Coverage	100%	Excelente
Method Coverage	100%	Excelente
Class Coverage	100%	Excelente

Tabla 3 Resumen de Cobertura

13. Evaluación de Calidad:

Velocidad: tests < 100ms

Determinismo: resultados siempre iguales



Mantenibilidad: código limpio, fácil de actualizar

Independencia: pruebas no dependen del orden

Validación completa: todos los requisitos cubiertos, incluyendo casos de borde y entradas no válidas

14. Documentación Javadoc:

- Se generan con:

```
javadoc -d docs src/main/java/com/ejemplo/*.java
```

- Incluye:

Descripción de la clase

Parámetros y valores de retorno de cada método

Posibles excepciones o errores de validación

Package com.corhuila.calidad

Class Convertidor

java.lang.Object[Ⓢ]
com.corhuila.calidad.Convertidor

public class **Convertidor**
extends Object[Ⓢ]

Clase encargada de realizar conversiones de temperatura entre grados Celsius y Fahrenheit.

Soporta valores negativos y precisión decimal.

Constructor Summary

Constructor	Description
convertidor()	

Method Summary

Modifier and Type	Method	Description
double	celsiusFahrenheit(double c)	Convierte una temperatura de grados Celsius a Fahrenheit.
boolean	esNumerico(String [Ⓢ] input)	Verifica si una cadena de texto representa un valor numérico válido.
double	fahrenheitACelsius(double f)	Convierte una temperatura de grados Fahrenheit a Celsius.

Methods inherited from class java.lang.Object[Ⓢ]

clone[Ⓢ], equals[Ⓢ], finalize[Ⓢ], getClass[Ⓢ], hashCode[Ⓢ], notify[Ⓢ], notifyAll[Ⓢ], toString[Ⓢ], wait[Ⓢ], wait[Ⓢ], wait[Ⓢ]

Ilustración 3 Documentación Javadoc



- JavaDoc

JavaDoc es una herramienta utilizada para generar documentación del código fuente en Java a partir de comentarios estructurados.

¿Para qué se usa?

- Documentar clases y métodos
- Explicar parámetros y valores de retorno
- Mejorar la mantenibilidad del código
- Facilitar la comprensión para otros desarrolladores

¿Cómo se hace?

Se utilizan comentarios especiales antes de clases o métodos:

```
/**  
 * Convierte grados Celsius a Fahrenheit.  
 *  
 * @param c temperatura en Celsius  
 * @return temperatura convertida a Fahrenheit  
 */  
public double celsiusAFahrenheit(double c) {  
    return (c * 9.0 / 5.0) + 32;  
}
```

15. Conclusiones:

- La implementación del sistema de conversión de temperaturas permitió aplicar de forma práctica los principios de calidad de software, evidenciando que el uso de pruebas unitarias con JUnit y métricas de cobertura como JaCoCo garantiza un comportamiento correcto, verificable y confiable del sistema desde etapas tempranas.



- La integración de herramientas y buenas prácticas, como Maven para la gestión del proyecto y JavaDoc para la documentación, contribuyó significativamente a la organización, mantenibilidad y comprensión del código, facilitando su evolución y reutilización.
- En conjunto, el uso de pruebas unitarias, medición de cobertura y documentación técnica demuestra ser fundamental para desarrollar software robusto, eficiente y de alta calidad, permitiendo detectar errores oportunamente y aumentar la confiabilidad del producto final.

16. Referencias:

- Eclipse Foundation. (2023). *JaCoCo Java code coverage library*. <https://www.jacoco.org>
- International Organization for Standardization. (2011). *ISO/IEC 25010: Systems and software quality models*. ISO.
- JUnit Team. (2023). *JUnit 5 user guide*. <https://junit.org>
- Oracle Corporation. (2023). *Java documentation (JavaDoc tool)*. <https://docs.oracle.com>
- Pressman, R. S. (2010). *Ingeniería del software: Un enfoque práctico* (7.ª ed.). McGraw-Hill.